

数理工学モデル化演習 第 10 回レポート

今村隼人

2026-06-21

1. 設定

問題 1 では、Erlang 分布の期待値を 20 に固定し、shape パラメータ y を 1, 2, 4, 8, 16, 32, 64, 128 と変化させた。Erlang 分布の期待値は $\frac{y}{x}$ であるため、rate パラメータを $x = \frac{y}{20}$ とした。

問題 2 と問題 3 では、サービス時間の期待値を 1 に固定するため $x = y$ とした。問題 2 では到着率を $\lambda = 0.5$ 、shape パラメータを $y = 1, 2, 4$ とした。問題 3 では到着率を $\lambda = 0.1, 0.2, \dots, 0.8$ とした。乱数シードは 2026 に固定し、各シミュレーションでは最初の 10000 イベントを 過渡状態として平均値の計算から除外した。

2. 結果

2.1. 問題 1

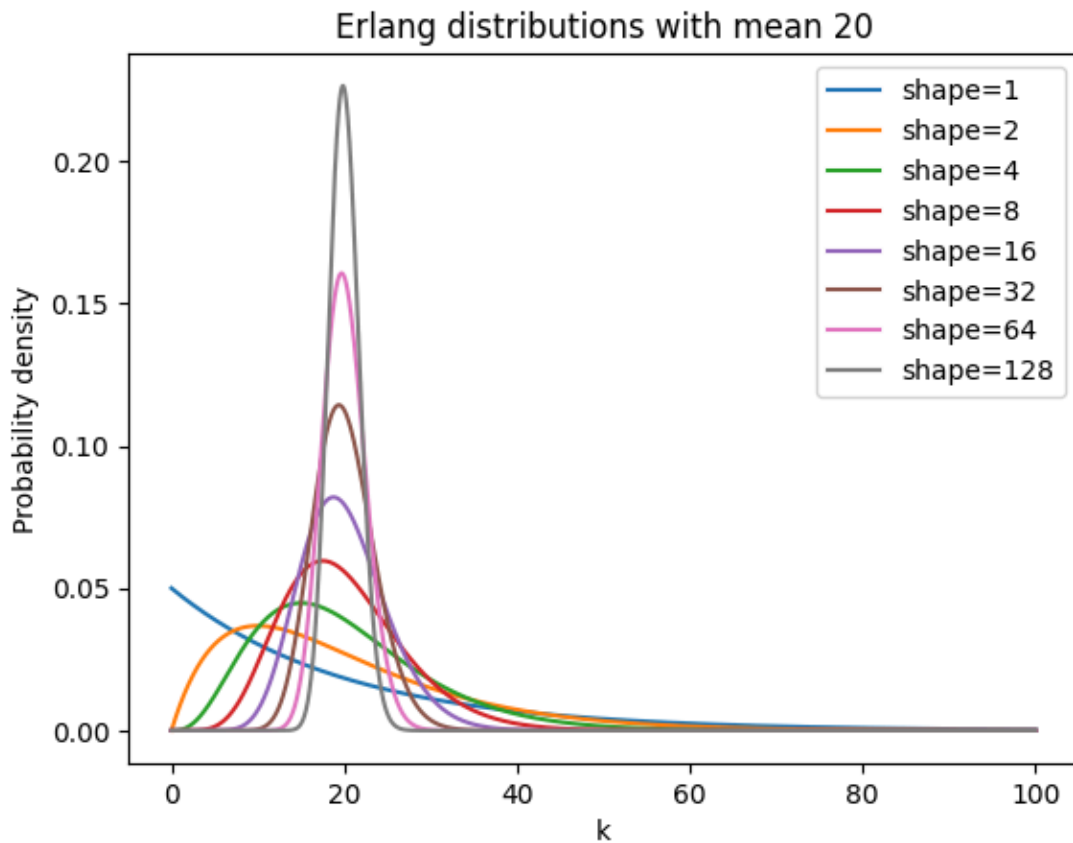


Figure 1: 期待値を 20 に固定した Erlang 分布の確率密度

shape パラメータが 1 のときは指数分布であり、 $k = 0$ で最大となって右側に長い裾を持つ。shape パラメータを大きくすると、分布の山は期待値 20 の周辺へ移動し、幅が狭くなった。Erlang 分布の分散は $\frac{y}{x^2} = \frac{400}{y}$ であるため、期待値を固定した場合でも shape パラメータを大きくするとばらつきが小さくなることが確認できる。

2.2. 問題 2

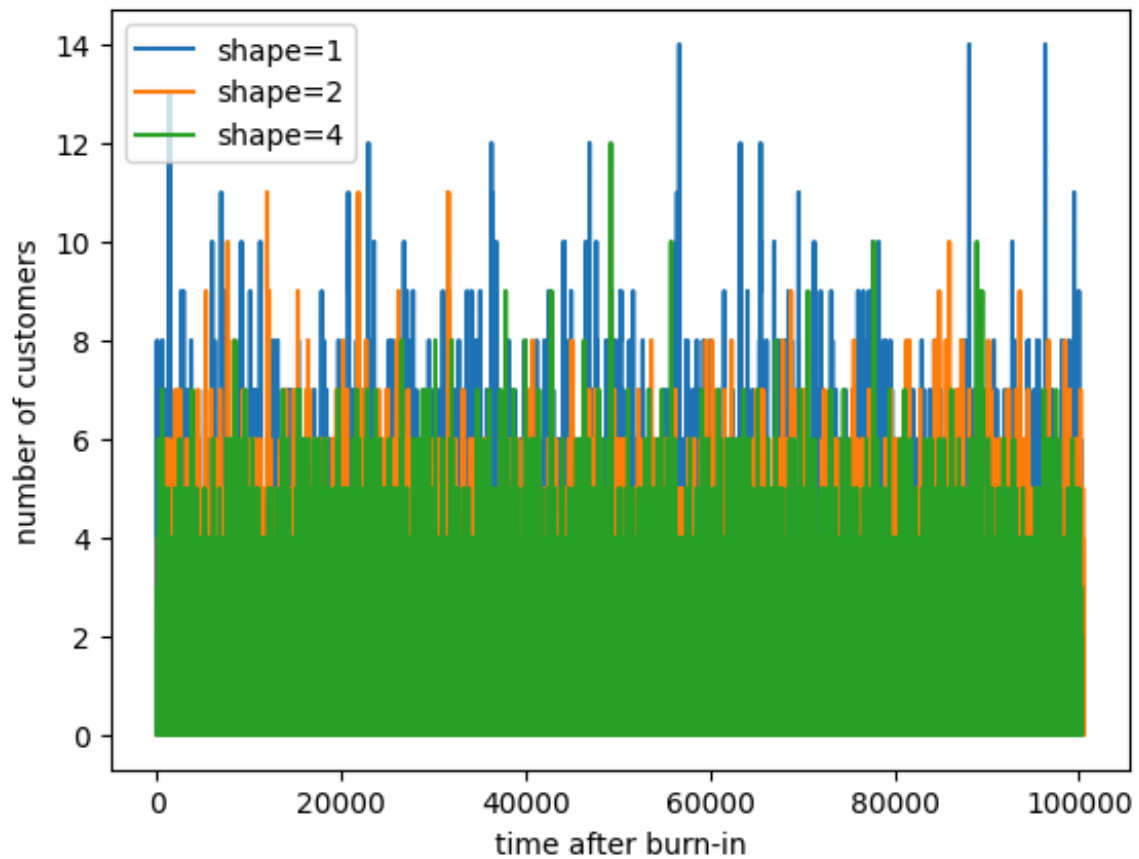


Figure 2: shape パラメータを 1、2、4 とした M/G/1 の系内客数の時間発展

いずれの場合も系内客数は到着によって 1 増加し、サービス完了によって 1 減少する。shape パラメータが大きいほどサービス時間の分散が小さくなるため、長いサービスによって待ち行列が大きく伸びる現象が起こりにくくなった。

2.3. 問題 3

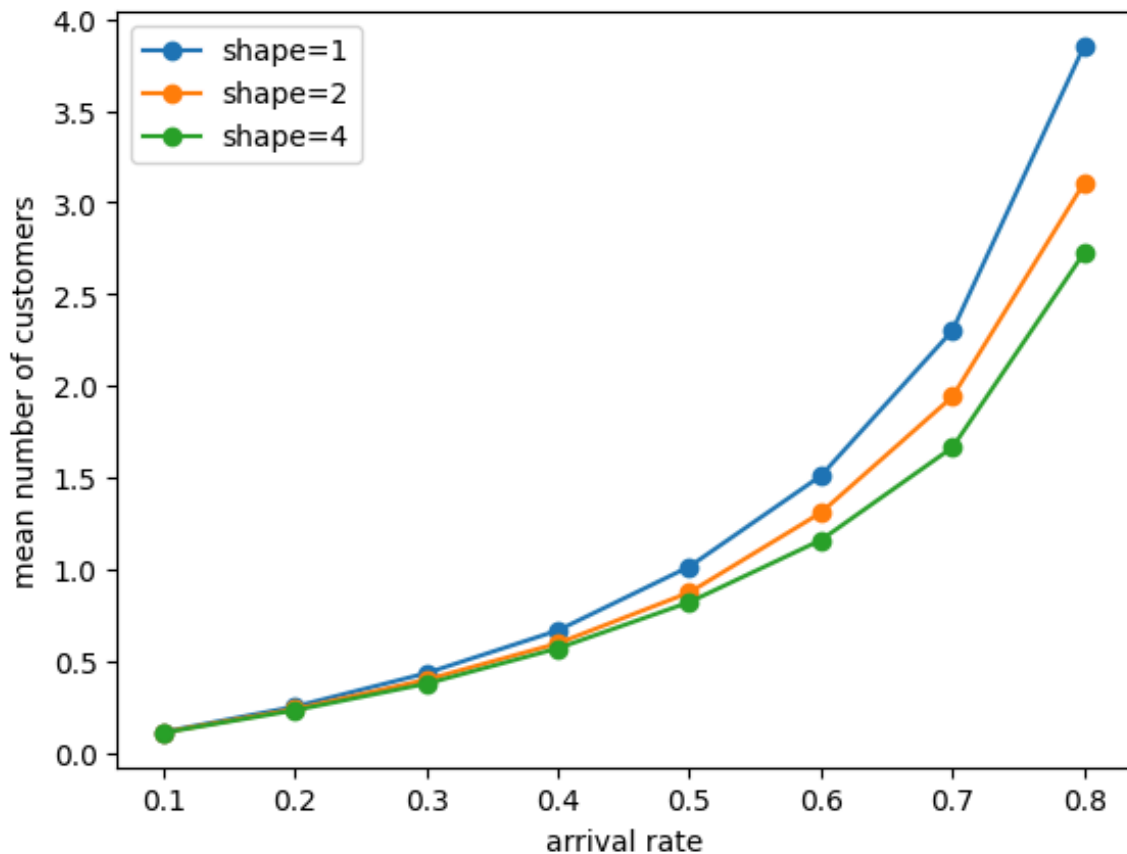


Figure 3: 到着率と shape パラメータを変えたときの平均系内客数

到着率が大きくなるほど平均系内客数は増加した。また、同じ到着率では shape パラメータが大きいほど平均系内客数は小さくなった。特に到着率が 1 に近づくと平均系内客数の増加が大きくなり、安定条件 $\lambda < 1$ の境界に近づく影響が現れた。

3. 考察

M/G/1 における平均待ち時間は Pollaczek–Khintchine の公式

$$E[W] = \frac{\lambda E[S^2]}{2(1 - \lambda E[S])}$$

で与えられる。今回の設定では $E[S] = 1$ であり、shape パラメータを y とすると $\text{Var}(S) = \frac{1}{y}$ 、 $E[S^2] = 1 + \frac{1}{y}$ となる。したがって、同じ到着率では shape パラメータが大きいほど平均待ち時間と平均系内客数が小さくなる。問題 2 と問題 3 のシミュレーション結果は、この関係と整合している。

4. 社会実装（問題 4）

M/G/1 の結果は、窓口、製造工程、通信処理など、1つの処理設備へ要求が到着する系に適用できる。平均処理時間が同じでも、処理時間のばらつきが大きいと平均待ち時間は長くなる。そのため、作業手順の標準化、処理内容ごとの窓口分離、通信パケット長の均一化などによって処理時間の分散を小さくすれば、設備を増やさずに待ち時間を短縮できると考えられる。

5. 演習の感想（問題 5）

3回の演習を通して、待ち行列では到着率とサービス率だけでなく、サーバ数、系の容量、サービス時間の分散が性能に影響することを確認できた。理論式だけでなく、イベントごとに状態を更新するシミュレーションを作ることで、系内客数が時間とともに変化し、定常状態の分布や平均値が得られる過程を具体的に理解

できた。一方で、イベント間隔や残余時間の更新を 誤ると別の確率過程になるため、各変数が表す時間を明確にする必要があると感じた。

6. 発展問題（問題 6）

G/M/1 では、到着間隔を Erlang 分布から生成し、サービス時間を指数分布から生成する。最初の実装ではイベントごとに Erlang 乱数を生成し直していたため、サービス完了が先に起きたときに、次の到着までの残り時間が失われていた。また、待ち客がいる場合の新しいサービス時間を Erlang 分布から生成しており、G/M/1 の定義と一致していなかった。

修正版では `next_arrival` と `residual_service` を別々に保持し、小さい方を次のイベントまでの時間とした。到着が先ならサービス残余時間を、サービス完了が先なら到着残余時間を経過時間だけ減らす。サービス時間は常に指数分布から生成する。掲載した実行例では、到着間隔を rate 2、shape 2 の Erlang 分布、サービス時間を rate 1.25 の指数分布とした。平均到着間隔は 1、平均サービス時間は 0.8 であり、利用率は 0.8 となるため安定条件を満たす。

7. ソースコード

7.1. 共通部分と M/G/1 シミュレーション

```
import math
from collections import deque

import matplotlib.pyplot as plt
import numpy as np
import numpy.random as random

random.seed(2026)

def exp(rate):
    return -math.log(random.random()) / rate

def erlang(rate, shape):
    value = 0
    for _ in range(shape):
        value += exp(rate)
    return value

def simulation_mg1_erlang(arrival_rate, service_rate, shape):
    customers = 0
    time = 0.0
    measured_time = 0.0
    area = 0.0
    next_arrival = exp(arrival_rate)
    residual_service = math.inf
    arrival_times = deque()
    sojourn_times = []
    customer_counts = []
    event_times = []
    burn_in = 10000

    for state in range(110000):
        customers_before = customers

        if next_arrival <= residual_service:
            interval = next_arrival
```

```

        if customers > 0:
            residual_service -= interval
            time += interval
            customers += 1
            arrival_times.append(time)
            next_arrival = exp(arrival_rate)
            if customers == 1:
                residual_service = erlang(service_rate, shape)
        else:
            interval = residual_service
            next_arrival -= interval
            time += interval
            customers -= 1
            arrived_at = arrival_times.popleft()
            if state >= burn_in:
                sojourn_times.append(time - arrived_at)
            residual_service = (
                erlang(service_rate, shape) if customers > 0 else math.inf
            )

    if state >= burn_in:
        measured_time += interval
        area += interval * customers_before
        customer_counts.append(customers)
        event_times.append(measured_time)

mean_customers = area / measured_time
return mean_customers, customer_counts, event_times, sojourn_times

```

7.2. 問題 1

```

for i in range(8):
    y = 2 ** i
    x = y / 20
    k = np.linspace(0,100,10001)
    p = []

    for j in k:
        pdf = ((x**y)/math.factorial(y-1))*(j**(y-1))*np.exp(-x*j)
        p.append(pdf)

plt.plot(k,p,label=f"shape={y}")

plt.xlabel("k")
plt.ylabel("Probability density")
plt.title("Erlang distributions with mean 20")
plt.legend()
plt.show()

```

7.3. 問題 2

```

for shape in (1, 2, 4):
    mean_customers, customer_counts, event_times, sojourn_times = (
        simulation_mgl_erlang(0.5, shape, shape)
    )
    plt.plot(event_times, customer_counts, label=f"shape={shape}")

plt.xlabel("time after burn-in")
plt.ylabel("number of customers")
plt.legend()
plt.show()

```

7.4. 問題 3

```
arrival_rates = np.arange(0.1, 0.9, 0.1)

for shape in (1, 2, 4):
    mean_customer_list = []
    for arrival_rate in arrival_rates:
        mean_customers, _, _, _ = simulation_mgl_erlang(
            arrival_rate, shape, shape
        )
        mean_customer_list.append(mean_customers)
    plt.plot(arrival_rates, mean_customer_list, marker="o", label=f"shape={shape}")

plt.xlabel("arrival rate")
plt.ylabel("mean number of customers")
plt.legend()
plt.show()
```

7.5. 問題 6

```
def simulation_gm1_erlang(service_rate, arrival_rate, shape):
    customers = 0
    time = 0.0
    measured_time = 0.0
    area = 0.0
    next_arrival = erlang(arrival_rate, shape)
    residual_service = math.inf
    arrival_times = deque()
    sojourn_times = []
    customer_counts = []
    event_times = []
    burn_in = 10000

    for state in range(110000):
        customers_before = customers

        if next_arrival <= residual_service:
            interval = next_arrival
            if customers > 0:
                residual_service -= interval
            time += interval
            customers += 1
            arrival_times.append(time)
            next_arrival = erlang(arrival_rate, shape)
            if customers == 1:
                residual_service = exp(service_rate)
        else:
            interval = residual_service
            next_arrival -= interval
            time += interval
            customers -= 1
            arrived_at = arrival_times.popleft()
            if state >= burn_in:
                sojourn_times.append(time - arrived_at)
            residual_service = exp(service_rate) if customers > 0 else math.inf

        if state >= burn_in:
            measured_time += interval
            area += interval * customers_before
            customer_counts.append(customers)
            event_times.append(measured_time)
```

```
mean_customers = area / measured_time
return mean_customers, customer_counts, event_times, sojourn_times

mean_customers, customer_counts, event_times, sojourn_times = (
    simulation_gm1_erlang(1.25, 2, 2)
)
plt.plot(event_times, customer_counts)
plt.xlabel("time after burn-in")
plt.ylabel("number of customers")
plt.show()
```